SPE UMaT Chapter Website

Backend & Infrastructure Developer Guide

Supabase: database, RLS, Edge Functions, membership IDs and event check-in

Project	SPE UMaT Chapter website
Document	Backend and Infrastructure Developer Guide
Version	1.0
Audience	Backend and infrastructure developer
Companion	Frontend Developer Guide v1.0. Sections 4 and 17, and the appendices, are identical in both guides

1. Purpose and scope

This document is the complete specification for the backend and infrastructure of the SPE UMaT Chapter website. It is written for the backend developer and is to be followed as written. The frontend developer works from the companion Frontend Developer Guide. Both guides contain the same API contract (section 4), database schema (Appendix A) and rulings (section 17), so the two codebases will match without further discussion.

You own the Supabase project: the database, Row Level Security, Edge Functions, Storage, Auth configuration, membership ID generation, the event QR check-in engine, the merit ledger, email, scheduled jobs and deployment. You hold every secret and every rule that must not be trusted to the client. Members have no accounts, so the public Edge Functions must defend themselves against abuse.

Outside the backend's responsibility

- Pages, forms, QR image rendering, client-side CSV and Excel export, and menu hiding by role. These belong to the frontend developer.

1.1 Requirement traceability

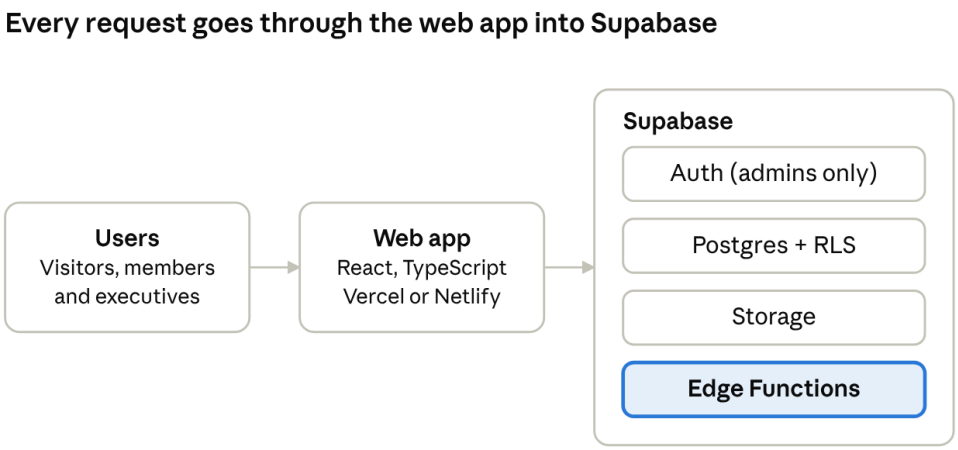
#	Committee requirement	Where it is covered in this guide
1	Homepage	5, 6 (public reads, views)
2	About page	5, 6
3	Website research and inspiration	Frontend only
4	Member registration and 6-digit membership ID	7.3, 8
5	Merit lookup by membership ID	7.3, 10
6	Administrative section and registrations export	6, 7.4
7	News, media and content management	6, 7.4
8	Merit Tracker and event QR check-in	7.3, 9, 10

2. Governing rules

Both developers work against the same contract. These five rules do not change without both developers agreeing.

1. **No accounts for visitors or members.** Their three actions (register, look up merit, check in) go through Edge Function endpoints only. The frontend never reads the `members` table directly.
2. **Only admins sign in**, through Supabase Auth. Admin screens read and write data under Row Level Security (RLS).
3. **The backend owns every secret** (service role key, CAPTCHA secret, Resend key, device-hash pepper). The frontend holds only the public URL and anon key.
4. **The schema in the appendix is the single source of truth.** Any change is agreed by both developers before merging.
5. **Validation and role checks on the client are UX only.** Server-side validation in the Edge Functions and RLS in the database are the real enforcement.

Architecture



Every request goes through the web app into Supabase: Auth (admins only), Postgres with RLS, Storage and Edge Functions. Edge Functions run with the service role key and bypass RLS, so each function enforces its own rules in code.

3. Platform setup

3.1 Stack

Layer	Choice
Database	PostgreSQL on Supabase (production on the Pro plan; a separate staging project)
Auth	Supabase Auth, admins only, with RLS
Functions	Supabase Edge Functions (Deno, TypeScript), `@supabase/supabase-js` v2, Zod
Storage	Supabase Storage, one public bucket `site-media`
Bot protection	Cloudflare Turnstile, verified server-side
Email	Resend API for application email; Resend SMTP for Supabase Auth email
Scheduling	`pg_cron`

Layer	Choice
Deployment	Supabase CLI from GitHub Actions. The frontend deploys through Vercel's GitHub integration

3.2 Supabase Auth configuration

Setting	Value
Public sign-ups	Disabled. Admins exist only through `invite_admin` and the bootstrap in 3.5
Site URL	Production site URL. Staging project uses the staging URL
Redirect URLs	`<site>/admin/accept-invite` and `<site>/admin/reset-password` for production, staging and `http://localhost:5173`
JWT expiry	3600 seconds. Refresh token rotation on, reuse interval 10 seconds
Password policy	Minimum 12 characters, letters and digits required. Leaked-password protection on
Custom SMTP	Resend SMTP: host `smtp.resend.com`, port 465, user `resend`, password the Resend API key, sender `EMAIL_FROM`. The default Supabase mailer is rate-limited and must not be used in production
Email templates	Invite and password-reset templates carry the chapter name and the line "This link works once and expires."
Access token hook	Enabled, pointing at `public.custom_access_token_hook` (section 6.2)

3.3 Secrets

```
# set per project with: supabase secrets set --env-file supabase/.env.<staging|production>
CAPTCHA_SECRET_KEY=        # Turnstile secret
RESEND_API_KEY=
EMAIL_FROM=                # e.g. no-reply@<chapter domain>; staging: onboarding@resend.dev
DEVICE_HASH_PEPPER=        # 32+ random bytes, hex. Hashes devices and IPs
CODE_HASH_PEPPER=          # 32+ random bytes, hex. Hashes email verification codes
WEBHOOK_SECRET=            # shared with the contact_messages database webhook
SITE_URL=                  # https://<site domain>
ALLOWED_ORIGINS=           # comma-separated; production site URL (staging adds localhost and
previews)
# Provided automatically by Supabase to Edge Functions:
# SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY
```

Generate peppers with `openssl rand -hex 32`. Each environment has its own values. Secrets are never committed, logged or sent to the frontend. `.env.*` files are in `.gitignore`.

3.4 Repository layout

```
supabase/
  config.toml              # [functions.<name>] verify_jwt = false for the public functions
  migrations/              # 0001_schema, 0002_functions, 0003_rls, 0004_views, 0005_storage,
                           # 0006_cron, 0007_seed
  seed/                    # staging seed script
  functions/
    _shared/               # cors, response, errors, validate, ratelimit, hash, geo,
                           # qrtoken, normalize, turnstile, email, audit, adminAuth, db
    register_member/  verify_email/  resend_code/  merit_lookup/
    start_check_in/   check_in/      submit_contact/  send_contact_notification/
    qr_token/  set_checkin_state/  reset_event_secret/  manual_check_in/
    remove_check_in/  adjust_merit/  log_export/  delete_event/
    delete_news_post/  delete_media_item/  delete_member/
    list_admins/  invite_admin/  set_user_role/
```

```
# supabase/config.toml  (public functions)
[functions.register_member]            verify_jwt = false
[functions.verify_email]               verify_jwt = false
[functions.resend_code]                verify_jwt = false
[functions.merit_lookup]               verify_jwt = false
[functions.start_check_in]             verify_jwt = false
[functions.check_in]                   verify_jwt = false
[functions.submit_contact]             verify_jwt = false
[functions.send_contact_notification]  verify_jwt = false   # protected by WEBHOOK_SECRET
```

3.5 Bootstrap of the first Technical Head

1. In the Supabase dashboard open Authentication, Users, Add user. Use the email from prerequisite P7, tick “Auto confirm”, and set a long temporary password.
2. Run the SQL below once in each project.
3. Tell the Technical Head to sign in at `/admin/login` and change the password through `/admin/reset-password`. From then on all admins are invited from the Admins screen.

```
insert into user_roles (user_id, role)
select id, 'technical_head' from auth.users where email = '<email from P7>';
```

3.6 Seed data

- **All environments:** `site_content` rows for `home_intro`, `about_mission`, `about_vision`, `about_benefits` and `contact_info` with placeholder Markdown the Communications Head replaces.
- **Staging only ({{supabase/seed}}):** 3 published events (one past, two upcoming) and 1 draft event with venue set, 4 news posts, 12 media items across categories, 10 test members with IDs `100001` to `100010` (some with leaderboard opt-in), merit rows for them, and three test admins (one per role) with known passwords documented to the frontend developer.

4. API contract

> **Authoritative** This section is identical in the Frontend Guide. Implement it exactly. A change requires a written amendment to both guides.

4.1 Conventions

All functions follow the same conventions. They are fixed and apply to both codebases.

Item	Rule
URL	`https://<project-ref>.supabase.co/functions/v1/<function_name>`
Method and body	`POST` with a JSON body, `Content-Type: application/json`. Functions with no request fields take `{}`
Headers, public functions	`apikey: <anon key>`. No `Authorization` header. These functions are deployed with JWT verification off (`--no-verify-jwt`) because visitors have no session
Headers, admin functions	`apikey: <anon key>` and `Authorization: Bearer <admin access token>` from the current Supabase session. JWT verification stays on, and each function also checks the role in code
Success response	HTTP 200. `{ "ok": true, "data": { ... } }`. Functions with no response fields return `"data": {}`

Item	Rule
Error response	HTTP status from the error table. `{ "ok": false, "error": { "code": "...", "message": "...", "details": [...], "retry_after_seconds": 30 } }`. `details` and `retry_after_seconds` appear only when relevant
What the client switches on	The `error.code` value, never the HTTP status or the message text
Field names	`snake_case` in every request and response
Timestamps	ISO 8601 strings in UTC. The site displays them in the `Africa/Accra` time zone
CORS	Allowed origins are the production site URL, plus `http://localhost:5173` and the Vercel preview URL on staging only. `OPTIONS` requests are answered by every function
Timeout	The client aborts a call after 15 seconds and shows the `SERVER_ERROR` message

4.2 Error codes

Return exactly these codes and HTTP statuses. The last column is the message text the frontend displays, so your `error.message` values should match it.

Code	HTTP	Raised when	Message shown to the user
`VALIDATION_ERROR`	400	Input fails validation. `error.details` lists `field` and `message` pairs	Show each message beside its field
`CAPTCHA_FAILED`	400	CAPTCHA token missing or rejected	Please complete the verification and try again.
`CODE_INVALID`	400	Wrong email verification code	That code is not correct.
`LOCATION_DENIED`	400	No position supplied, permission denied, or accuracy worse than 100 m	Location is required. Allow location for this site, then scan the QR code again.
`UNAUTHORIZED`	401	Admin call without a valid session	Redirect to the admin login page
`FORBIDDEN`	403	Signed-in admin lacks the required role	You do not have permission to do this.
`CHECKIN_CLOSED`	403	Event unpublished, outside the check-in window, or check-in closed by an admin	Check-in is closed for this event.
`OUTSIDE_AREA`	403	Phone is farther from the venue than the event radius	You do not appear to be at the event. Move closer and try again.
`ID_NOT_FOUND`	404	No member has this ID. One generic response for every unknown ID	ID not found.
`NOT_FOUND`	404	Admin call refers to a record that does not exist	This item no longer exists.
`DUPLICATE_REGISTRATION`	409	Email or phone already registered	This email or phone number is already registered. Please contact the chapter executives.
`ALREADY_CHECKED_IN`	409	Member already checked in to this event. No extra points	You are already checked in to this event.
`DEVICE_ALREADY_USED`	409	This phone already checked someone in for this event	This phone has already been used to check in for this event.

Code	HTTP	Raised when	Message shown to the user
`EVENT_HAS_CHECKINS`	409	Admin tried to delete an event that has check-ins	This event has check-ins and cannot be deleted. Unpublish it instead.
`LAST_TECHNICAL_HEAD`	409	Attempt to remove the last Technical Head, or the caller's own Technical Head role	At least one Technical Head must remain.
`CODE_EXPIRED`	410	Verification code older than 10 minutes, or no pending registration	That code has expired. Request a new one or register again.
`QR_EXPIRED`	410	QR token outside its life, invalid signature, or event secret reset	This QR code has expired. Scan the screen again.
`SESSION_INVALID`	410	Check-in session missing, used, expired (3 minutes), from another device, or over 5 attempts	Your check-in session has ended. Scan the QR code again.
`RATE_LIMITED`	429	Too many requests. `error.retry_after_seconds` is set	Too many attempts. Please wait and try again.
`TOO_MANY_ATTEMPTS`	429	More than 5 wrong verification codes for one registration	Too many wrong codes. Please register again.
`SERVER_ERROR`	500	Unexpected failure	Something went wrong. Please try again.

4.3 Public functions (no login)

register_member

Access: Public. JWT verification off. Validates the registration, rejects duplicates, stores a pending registration and emails a 6-digit verification code.

Request field	Type	Rules
`first_name`	string	Required. 1 to 50 characters after trimming
`last_name`	string	Required. 1 to 50 characters after trimming
`programme`	string	Required. 2 to 100 characters. A value from the programme list, or free text when "Other" is chosen
`year_of_study`	string	Required. One of `Year 1`, `Year 2`, `Year 3`, `Year 4`, `Postgraduate`
`email`	string	Required. Valid email, maximum 254 characters. Stored in lower case
`phone`	string	Required. Ghana format (`0XXXXXXXXX`, `233XXXXXXXXX`, `+233XXXXXXXXX`) or international E.164 (`+` and 8 to 15 digits). Stored as E.164
`show_on_leaderboard`	boolean	Optional. Default `false`
`captcha_token`	string	Required. Token from the Turnstile widget
`website`	string	Honeypot. Always send an empty string. A non-empty value is discarded silently

Response field	Type	Description
`email`	string	The normalised email the code was sent to
`expires_in_seconds`	integer	600

Response field	Type	Description
resend_available_in_seconds	integer	60

Errors: VALIDATION_ERROR, CAPTCHA_FAILED, DUPLICATE_REGISTRATION, RATE_LIMITED.

verify_email

Access: Public. JWT verification off. Checks the emailed code, creates the member, generates the membership ID, returns it once and emails it.

Request field	Type	Rules
email	string	Required. The email used to register
code	string	Required. Exactly 6 digits

Response field	Type	Description
member_code	string	The new 6-digit membership ID. Returned only by this call
first_name	string	—

Errors: VALIDATION_ERROR, CODE_INVALID, CODE_EXPIRED, TOO_MANY_ATTEMPTS, DUPLICATE_REGISTRATION, RATE_LIMITED.

resend_code

Access: Public. JWT verification off. Sends a new verification code for a pending registration. Allowed after a 60-second cooldown and at most 3 times per registration.

Request field	Type	Rules
email	string	Required

Response field	Type	Description
expires_in_seconds	integer	600
resend_available_in_seconds	integer	60

Errors: VALIDATION_ERROR, CODE_EXPIRED (no pending registration), RATE_LIMITED.

merit_lookup

Access: Public. JWT verification off. Returns a member's first name, total points and merit history. Returns no contact details.

Request field	Type	Rules
member_code	string	Required. Exactly 6 digits, first digit 1 to 9

Response field	Type	Description
first_name	string	—
total_points	integer	Sum of all ledger rows

Response field	Type	Description
attendance	array	Each item: event_title string, event_date ISO timestamp, points integer, checked_in_at ISO timestamp. Newest first
adjustments	array	Each item: points integer, reason string, created_at ISO timestamp. Newest first

Errors: VALIDATION_ERROR, ID_NOT_FOUND, RATE_LIMITED.

start_check_in

Access: Public. JWT verification off. Validates the scanned QR token, the phone's location and the event window, then opens a one-time check-in session bound to the phone.

Request field	Type	Rules
event_token	string	Required. The token from the QR address
device_token	string	Required. UUID generated by the browser and kept in local storage (see Frontend section 7.5)
location	object	Required. lat number (-90 to 90), lng number (-180 to 180), accuracy number in metres (greater than 0)

Response field	Type	Description
session_id	uuid	Valid once, for 3 minutes, from this device only
expires_at	string	ISO timestamp, 3 minutes after the scan
event_title	string	—
points	integer	Points this check-in will award

Errors: VALIDATION_ERROR, QR_EXPIRED, CHECKIN_CLOSED, LOCATION_DENIED, OUTSIDE_AREA, RATE_LIMITED.

check_in

Access: Public. JWT verification off. Completes a check-in: validates the session, device, location, event window and member ID, then records attendance and awards points in one transaction.

Request field	Type	Rules
session_id	uuid	Required
member_code	string	Required. Exactly 6 digits, first digit 1 to 9
device_token	string	Required. UUID generated by the browser and kept in local storage (see Frontend section 7.5)
location	object	Required. lat number (-90 to 90), lng number (-180 to 180), accuracy number in metres (greater than 0)

Response field	Type	Description
first_name	string	—
points_awarded	integer	—
total_points	integer	New total
event_title	string	—

Errors: `VALIDATION_ERROR`, `SESSION_INVALID`, `CHECKIN_CLOSED`, `LOCATION_DENIED`, `OUTSIDE_AREA`, `ID_NOT_FOUND`, `ALREADY_CHECKED_IN`, `DEVICE_ALREADY_USED`, `RATE_LIMITED`.

submit_contact

Access: Public. JWT verification off. Stores a Contact Us message and notifies the executives.

Request field	Type	Rules
`name`	string	Required. 2 to 100 characters
`email`	string	Required. Valid email
`message`	string	Required. 10 to 2000 characters
`captcha_token`	string	Required
`website`	string	Honeypot. Always send an empty string

Response: `data: {}`.

Errors: `VALIDATION_ERROR`, `CAPTCHA_FAILED`, `RATE_LIMITED`.

4.4 Admin functions (admin session required)

qr_token

Access: Organization Head, Technical Head. Returns the current signed QR token for an event. Writes nothing to the database.

Request field	Type	Rules
`event_id`	uuid	Required

Response field	Type	Description
`token`	string	Place in `VITE_SITE_URL + "/checkin/" + token`
`refresh_in_seconds`	integer	Seconds until the next 30-second window starts. Call again then

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`.

set_checkin_state

Access: Organization Head, Technical Head. Opens or closes check-in for an event.

Request field	Type	Rules
`event_id`	uuid	Required
`is_active`	boolean	Required

Response field	Type	Description
`is_active`	boolean	—

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`.

reset_event_secret

Access: Organization Head, Technical Head. Replaces the event's secret. Every outstanding QR code and check-in session stops working immediately.

Request field	Type	Rules
`event_id`	uuid	Required

Response: `data: {}`.

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`.

manual_check_in

Access: Organization Head, Technical Head. Checks a member in by hand. Awards the event's points. No location or device check.

Request field	Type	Rules
`event_id`	uuid	Required
`member_code`	string	Required. Exactly 6 digits, first digit 1 to 9

Response field	Type	Description
`first_name`	string	—
`points_awarded`	integer	—
`total_points`	integer	—

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`, `ID_NOT_FOUND`, `ALREADY_CHECKED_IN`.

remove_check_in

Access: Organization Head, Technical Head. Removes a check-in and writes a reversing points row.

Request field	Type	Rules
`attendance_id`	uuid	Required
`reason`	string	Required. 3 to 200 characters

Response field	Type	Description
`total_points`	integer	The member's new total

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`.

adjust_merit

Access: Organization Head, Technical Head. Adds a manual points adjustment to the ledger.

Request field	Type	Rules
`member_code`	string	Required. Exactly 6 digits, first digit 1 to 9
`points`	integer	Required. Non-zero, from -100 to 100
`reason`	string	Required. 3 to 200 characters. Visible to the member in merit lookup

Response field	Type	Description
`total_points`	integer	—

Errors: `VALIDATION_ERROR`, `UNAUTHORIZED`, `FORBIDDEN`, `ID_NOT_FOUND`.

log_export

Access: Organization Head, Technical Head. Records a registrations export in the audit log. The frontend calls it before building the file and builds the file only if it succeeds.

Request field	Type	Rules
`format`	string	Required. `csv` or `xlsx`
`row_count`	integer	Required. Rows about to be exported

Response: `data: {}`.

Errors: `UNAUTHORIZED`, `FORBIDDEN`.

delete_event

Access: Organization Head, Technical Head. Deletes an event that has no check-ins, with its cover image and linked rows.

Request field	Type	Rules
`event_id`	uuid	Required

Response: `data: {}`.

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`, `EVENT_HAS_CHECKINS`.

delete_news_post

Access: Communications Head, Technical Head. Deletes a news post and its cover image.

Request field	Type	Rules
`post_id`	uuid	Required

Response: `data: {}`.

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`.

delete_media_item

Access: Communications Head, Technical Head. Deletes a media item and, for pictures, its file in Storage.

Request field	Type	Rules
`media_id`	uuid	Required

Response: `data: {}`.

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`.

delete_member

Access: Technical Head. Permanently deletes a member with their attendance and points. Used to honour a data-removal request.

Request field	Type	Rules
member_id	uuid	Required

Response: data: {}.

Errors: UNAUTHORIZED, FORBIDDEN, NOT_FOUND.

list_admins

Access: Technical Head. Lists admin accounts with their roles.

Request: empty object.

Response field	Type	Description
admins	array	Each item: user_id uuid, email string, roles array of role names, last_sign_in_at ISO timestamp or null

Errors: UNAUTHORIZED, FORBIDDEN.

invite_admin

Access: Technical Head. Creates an admin account, assigns a role and emails an invitation link to set a password.

Request field	Type	Rules
email	string	Required
role	string	Required. communications_head, organization_head or technical_head

Response field	Type	Description
user_id	uuid	—

Errors: VALIDATION_ERROR, UNAUTHORIZED, FORBIDDEN.

set_user_role

Access: Technical Head. Adds or removes one role for an admin. Removing all roles disables the admin's access.

Request field	Type	Rules
user_id	uuid	Required
role	string	Required. One of the three role names
action	string	Required. add or remove

Response field	Type	Description
roles	array	The admin's roles after the change

Errors: VALIDATION_ERROR, UNAUTHORIZED, FORBIDDEN, NOT_FOUND, LAST_TECHNICAL_HEAD.

TypeScript types for every request and response are in Appendix B. Write matching Zod schemas in _shared/validate and keep them in step with that file.

5. Database implementation

The tables are defined in Appendix A. This section adds everything the migrations must also contain. Constraint names are fixed because the functions map them to error codes.

5.1 Types, extensions and constraints

```
create extension if not exists pgcrypto with schema extensions;
create extension if not exists pg_cron;
create type app_role as enum ('communications_head','organization_head','technical_head');

-- named constraints (the functions rely on these names)
alter table members    add constraint members_email_key        unique (email);
alter table members    add constraint members_phone_key        unique (phone);
alter table members    add constraint members_member_code_key  unique (member_code);
alter table members    add constraint members_code_format      check (member_code ~ '^[1-9][0-
9]{5}$');
alter table members    add constraint members_year_check
  check (year_of_study in ('Year 1','Year 2','Year 3','Year 4','Postgraduate'));
alter table attendance add constraint attendance_event_member_uq unique (event_id, member_id);
alter table attendance add constraint attendance_event_device_uq unique (event_id, device_hash);
alter table events     add constraint events_radius_check      check (radius_m between 20 and 1000);

-- indexes
create index on merit_records (member_id);
create index on attendance (event_id, checked_in_at);
create index on attendance (event_id, ip_hash, ua_family);
create index on checkin_sessions (expires_at);
create index on email_verifications (expires_at);
create index on news_posts (is_published, published_at desc);
create index on events (is_published, starts_at);
create index on audit_logs (created_at desc);
```

Add a generic `set_updated_at` trigger to every table that has `updated_at`.

5.2 Event triggers

```
-- 1. every new event gets its own secret
create function events_create_token() returns trigger
language plpgsql security definer set search_path = public, extensions as $$
begin
  insert into event_checkin_tokens (event_id, secret)
  values (new.id, encode(gen_random_bytes(32), 'hex'));
  return new;
end $$;
create trigger trg_events_token after insert on events
  for each row execute function events_create_token();

-- 2. the Communications Head may not touch venue, radius, points or the check-in window
create function events_guard() returns trigger
language plpgsql security definer set search_path = public as $$
declare restricted boolean;
begin
  restricted := has_role('communications_head')
                and not (has_role('organization_head') or has_role('technical_head'));
  if restricted then
    if tg_op = 'INSERT' then
      new.points := 3; new.radius_m := 100;
      new.venue_lat := null; new.venue_lng := null;
      new.checkin_opens_at := null; new.checkin_closes_at := null;
    elsif new.points is distinct from old.points
       or new.radius_m is distinct from old.radius_m
       or new.venue_lat is distinct from old.venue_lat
       or new.venue_lng is distinct from old.venue_lng
       or new.checkin_opens_at is distinct from old.checkin_opens_at
       or new.checkin_closes_at is distinct from old.checkin_closes_at then
      raise exception 'FORBIDDEN';
```

```
    end if;
  end if;
  return new;
end $$;
create trigger trg_events_guard before insert or update on events
  for each row execute function events_guard();
```

The guard does not apply to service-role calls from Edge Functions, because `auth.uid()` is null there and `has_role` returns false.

5.3 Rate-limit function

```
create function rl_hit(p_key text, p_limit int, p_window_seconds int)
returns table (allowed boolean, retry_after int)
language plpgsql security definer set search_path = public as $$
declare v_count int; v_start timestamptz;
begin
  insert into rate_limits as r (key, window_start, count) values (p_key, now(), 1)
  on conflict (key) do update set
    window_start = case when r.window_start < now() - make_interval(secs => p_window_seconds)
                        then now() else r.window_start end,
    count        = case when r.window_start < now() - make_interval(secs => p_window_seconds)
                        then 1 else r.count + 1 end
  returning count, window_start into v_count, v_start;
  return query select v_count <= p_limit,
    greatest(1, ceil(extract(epoch from (v_start + make_interval(secs => p_window_seconds) -
now())))::int);
end $$;
revoke all on function rl_hit from public, anon, authenticated;
```

5.4 Views

```
create view events_public as
  select id, title, slug, description, starts_at, ends_at, location, cover_image_url, points
  from events where is_published;

create view leaderboard as
select display_name, total_points, rank() over (order by total_points desc)::int as rank
from (
  select m.first_name || ' ' || upper(left(m.last_name, 1)) || '.' as display_name,
         coalesce(sum(r.points), 0)::int as total_points
  from members m left join merit_records r on r.member_id = m.id
  where m.show_on_leaderboard
  group by m.id
  having coalesce(sum(r.points), 0) > 0
) t;

create view admin_registrations with (security_invoker = true) as
select m.id, m.first_name, m.last_name, m.email, m.phone, m.programme, m.year_of_study,
       m.member_code, m.show_on_leaderboard, m.created_at,
       coalesce(sum(r.points), 0)::int as total_points
from members m left join merit_records r on r.member_id = m.id
group by m.id;

create view admin_attendance with (security_invoker = true) as
select a.id as attendance_id, a.event_id, a.member_id, m.member_code, m.first_name, m.last_name,
       a.checked_in_at, a.method, a.distance_m, a.ip_hash, a.ua_family
from attendance a join members m on m.id = a.member_id;

create view admin_merit with (security_invoker = true) as
select r.id, r.member_id, m.member_code, m.first_name, m.last_name, r.event_id,
       e.title as event_title, r.points, r.source, r.reason, r.awarded_by, r.created_at
from merit_records r join members m on m.id = r.member_id
left join events e on e.id = r.event_id;
```

5.5 Check-in and ledger functions

Edge Functions cannot share a transaction across separate calls, so the writes that must succeed or fail together are Postgres functions called with `supabase.rpc` using the service role.

```
create function perform_check_in(
  p_session uuid, p_event uuid, p_member uuid,
  p_device_hash text, p_distance int, p_ip_hash text, p_ua text)
returns table (points_awarded int, total_points int)
language plpgsql security definer set search_path = public as $$
declare v_points int; v_att uuid;
begin
  update checkin_sessions set used_at = now()
   where id = p_session and used_at is null and expires_at > now();
  if not found then raise exception 'SESSION_INVALID'; end if;

  select points into v_points from events where id = p_event;

  insert into attendance (event_id, member_id, method, device_hash, distance_m, ip_hash, ua_family)
  values (p_event, p_member, 'qr', p_device_hash, p_distance, p_ip_hash, p_ua)
  returning id into v_att;

  insert into merit_records (member_id, event_id, attendance_id, points, source)
  values (p_member, p_event, v_att, v_points, 'event_checkin');

  return query select v_points,
    (select coalesce(sum(points), 0)::int from merit_records where member_id = p_member);
end $$;
revoke all on function perform_check_in from public, anon, authenticated;
```

The same pattern is used for the other multi-step writes. Create each as a security-definer function, executable only by the service role:

Function	Does, in one transaction
`perform_manual_check_in(event, member, admin)`	Inserts `attendance` with `method = manual` and null device, distance and hashes; inserts the `event_checkin` ledger row linked to it. Unique violations are raised as in `perform_check_in`
`perform_remove_check_in(attendance, reason, admin)`	Locks the attendance row; sums the points its ledger rows awarded; inserts a `check_in_reversal` row of the negative amount with the reason and admin; deletes the attendance row; returns the member's new total
`perform_delete_member(member)`	Deletes the member's `merit_records`, then `attendance`, then the `members` row
`member_total(member)`	Returns the sum of the member's ledger rows

6. Row Level Security and roles

6.1 Principle and role matrix

Visitors and members have no direct database access except reading published public content. Everything else goes through an Edge Function. Start from default deny: enable RLS on every table in `public`, revoke all table privileges from `anon` and `authenticated`, then grant only what the policies below need.

Admin screen	Communications Head	Organization Head	Technical Head
Overview (widgets limited to what the role can read)	Yes	Yes	Yes

Admin screen	Communications Head	Organization Head	Technical Head
Registrations, export and member deletion	No	Yes (no deletion)	Yes
Events: create, edit, publish, cover image	Yes	Yes	Yes
Events: venue, radius, points, check-in window, QR, close and reset check-in	No	Yes	Yes
Events: delete	No	Yes	Yes
News	Yes	No	Yes
Attendance	No	Yes	Yes
Merit Tracker and manual adjustments	No	Yes	Yes
Media and Content	Yes	No	Yes
Messages	Yes	No	Yes
Audit log	No	No	Yes
Admins (invite, roles)	No	No	Yes

6.2 Role helper and access token hook

Do not use `auth.role()` for application roles: Supabase returns only `anon` or `authenticated`. Roles live in `user_roles` and are checked with `has_role()`.

```
create function public.has_role(r app_role) returns boolean
language sql stable security definer set search_path = public as $$
  select exists (select 1 from user_roles where user_id = auth.uid() and role = r);
$$;

create function public.has_any_role() returns boolean
language sql stable security definer set search_path = public as $$
  select exists (select 1 from user_roles where user_id = auth.uid());
$$;

-- adds app_roles to every admin's access token, for the frontend to read
create function public.custom_access_token_hook(event jsonb) returns jsonb
language plpgsql stable as $$
declare claims jsonb; roles text[];
begin
  select coalesce(array_agg(role::text), '{}') into roles
    from public.user_roles where user_id = (event->>'user_id')::uuid;
  claims := jsonb_set(event->'claims', '{app_roles}', to_jsonb(roles));
  return jsonb_set(event, '{claims}', claims);
end $$;

grant usage on schema public to supabase_auth_admin;
grant execute on function public.custom_access_token_hook to supabase_auth_admin;
revoke execute on function public.custom_access_token_hook from authenticated, anon, public;
grant select on table public.user_roles to supabase_auth_admin;
create policy auth_admin_reads_roles on public.user_roles
  as permissive for select to supabase_auth_admin using (true);
```

Enable the hook in Authentication settings. A role change reaches the admin's token at the next refresh; `set_user_role` callers refresh their own session.

6.3 Policies

Table	anon	Admin access (authenticated)
site_content	select	update: Communications Head, Technical Head
events	none (public reads events_public)	select, insert, update: any admin role (guarded by events_guard). No delete
event_checkin_tokens	none	select: Organization Head, Technical Head. No other client access
news_posts	select where is_published	select, insert, update: Communications Head, Technical Head. No delete
media_items	select where is_published	select, insert, update: Communications Head, Technical Head. No delete
members	none	select: Organization Head, Technical Head
attendance, merit_records	none	select: Organization Head, Technical Head. No client writes
contact_messages	none	select, update: Communications Head, Technical Head. Inserts only by submit_contact
audit_logs	none	select: Technical Head. No client writes
user_roles, email_verifications, checkin_sessions, rate_limits	none	none (service role only)

```
alter table members enable row level security;     -- repeat for every table
revoke all on all tables in schema public from anon, authenticated;

grant select on events_public, leaderboard to anon, authenticated;
grant select on admin_registrations, admin_attendance, admin_merit to authenticated;

grant select on site_content, news_posts, media_items to anon;
create policy site_content_read on site_content for select to anon, authenticated using (true);
create policy news_public on news_posts for select to anon using (is_published);
create policy media_public on media_items for select to anon using (is_published);

grant select, update on site_content to authenticated;
create policy site_content_write on site_content for update to authenticated
  using (has_role('communications_head') or has_role('technical_head'))
  with check (has_role('communications_head') or has_role('technical_head'));

grant select, insert, update on events to authenticated;
create policy events_admin on events for all to authenticated
  using (has_any_role()) with check (has_any_role());       -- no delete grant

grant select, insert, update on news_posts, media_items to authenticated;
create policy news_admin on news_posts for all to authenticated
  using (has_role('communications_head') or has_role('technical_head'))
  with check (has_role('communications_head') or has_role('technical_head'));
-- media_items: same as news_posts

grant select on members, attendance, merit_records, event_checkin_tokens to authenticated;
create policy members_read on members for select to authenticated
  using (has_role('organization_head') or has_role('technical_head'));
-- attendance, merit_records, event_checkin_tokens: same condition

grant select, update (is_read) on contact_messages to authenticated;
create policy messages_admin on contact_messages for all to authenticated
  using (has_role('communications_head') or has_role('technical_head'));
```

```
grant select on audit_logs to authenticated;
create policy audit_read on audit_logs for select to authenticated using
(has_role('technical_head'));
```

Every table without a listed policy has none, so access is denied. `audit_logs` and `merit_records` receive no update or delete privilege for any role through the API. The functions never update or delete them, except the reversal pattern in section 10.

6.4 Storage

```
insert into storage.buckets (id, name, public, file_size_limit, allowed_mime_types)
values ('site-media', 'site-media', true, 5242880, array['image/jpeg','image/png','image/webp']);

create policy media_read on storage.objects for select using (bucket_id = 'site-media');

create policy media_events_write on storage.objects for insert to authenticated
  with check (bucket_id = 'site-media' and (storage.foldername(name))[1] = 'events' and
has_any_role());
create policy media_events_delete on storage.objects for delete to authenticated
  using (bucket_id = 'site-media' and (storage.foldername(name))[1] = 'events' and has_any_role());

create policy media_content_write on storage.objects for insert to authenticated
  with check (bucket_id = 'site-media' and (storage.foldername(name))[1] in ('news','gallery')
              and (has_role('communications_head') or has_role('technical_head')));
create policy media_content_delete on storage.objects for delete to authenticated
  using (bucket_id = 'site-media' and (storage.foldername(name))[1] in ('news','gallery')
         and (has_role('communications_head') or has_role('technical_head')));
```

The bucket is public, so it must contain only pictures intended to be public. Functions that delete rows remove the matching files with the service role.

7. Edge Function implementation

7.1 Rules for every function

- **Structure.** `Deno.serve`, handle `OPTIONS`, accept `POST` only, parse JSON, validate with Zod, run the logic, and return the response envelope from section 4.1 through `_shared/response`. Wrap everything in a try/catch that returns `SERVER_ERROR`.
- **CORS.** Reply with `Access-Control-Allow-Origin` only when the request origin is in `ALLOWED_ORIGINS`.
- **Client IP.** First value of `x-forwarded-for`. The hashed IP is `HMAC-SHA256(DEVICE_HASH_PEPPER, ip)` in hex. `ua_family` is a coarse label derived from the `User-Agent` (for example `Chrome Android`, `Safari iOS`, `Other`).
- **Rate limiting.** Call `rl_hit` for every key in 7.2 before doing any other work. On refusal return `RATE_LIMITED` with `retry_after_seconds`.
- **Generic errors.** An unknown member ID always returns `ID_NOT_FOUND` with the same body. No response reveals whether an email, phone or ID exists, except `DUPLICATE_REGISTRATION` as specified.
- **Logging.** Log only function name, request id, error code and duration. Never log request bodies, coordinates, tokens, codes, IDs, emails or phone numbers.
- **Admin functions.** Read the `Authorization` bearer token, resolve the user with `supabase.auth.getUser(jwt)`, load their roles from `user_roles`, and return `UNAUTHORIZED` or `FORBIDDEN` before anything else. After success write one `audit_logs` row (7.4).
- **Database writes.** Use the service-role client. Use only parameterised queries or the supabase-js query builder.
- **Time.** Use the database clock (`now()`) for expiry decisions wherever a database call is already being made.

7.2 Rate limits

These thresholds are final for launch. Keep them in `_shared/ratelimit.ts` as one table so they can be tuned after the first event without code changes elsewhere. Do not block by IP alone on the check-in functions: members share the campus network.

Function	Key	Limit	Window
`register_member`	IP	5	1 hour
	email	3	1 hour
`verify_email`	IP	20	1 hour
	email	10	1 hour
`resend_code`	IP	10	1 hour
	email	3 per registration, 60 seconds apart	stored on the pending row
`merit_lookup`	IP	30	10 minutes
	member ID	10	10 minutes
	IP, failed lookups only	10	10 minutes
`start_check_in`	device hash	10	10 minutes
	IP	300	10 minutes
`check_in`	device hash	10	10 minutes
	member ID	10	10 minutes
	IP	300	10 minutes
	session	5 attempts	life of the session
`submit_contact`	IP	3	1 hour
	email	5	1 day
Admin functions	admin user	120	1 minute

7.3 Public functions: processing rules

register_member

1. Rate-limit (IP, then email once parsed). Validate. If `website` is not empty, return a normal success response with `expires_in_seconds: 600` and do nothing else.
2. Verify `captcha_token` with Turnstile's `siteverify` endpoint and the caller's IP. Failure returns `CAPTCHA_FAILED`.
3. Normalise email and phone (section 8.1).
4. If `members` has a row with the same email or phone, return `DUPLICATE_REGISTRATION` without revealing the ID.
5. Generate a 6-digit code. Upsert the single `email_verifications` row for the email: `code_hash`, `registration_data`, `expires_at = now() + 10 minutes`, `attempts = 0`, `resend_count = 0`, `last_sent_at = now()`.
6. Send the verification email (section 11). If sending fails, return `SERVER_ERROR`.
7. Return `email`, `expires_in_seconds: 600`, `resend_available_in_seconds: 60`.

verify_email

1. Rate-limit. Validate. Load the pending row for the email. Missing or past `expires_at` returns `CODE_EXPIRED`.
2. Increment `attempts` in a single update that returns the new value. If it is above 5 return `TOO_MANY_ATTEMPTS`.
3. Compare `HMAC-SHA256(CODE_HASH_PEPPER, email + ":" + code)` with `code_hash` in constant time. A mismatch returns `CODE_INVALID`.
4. Generate the membership ID and insert the member with `email_verified_at = now()` (section 8.2). A unique violation on email or phone returns `DUPLICATE_REGISTRATION`.
5. Delete the pending row. Send the membership ID email (a failure is logged but does not fail the request).
6. Return `member_code` and `first_name`. This is the only response that ever contains a new ID.

resend_code

1. Rate-limit. Validate. Load the pending row; missing or expired returns `CODE_EXPIRED`.
2. If `resend_count` is 3 or `last_sent_at` is less than 60 seconds ago, return `RATE_LIMITED` with the seconds remaining.
3. Generate a new code, replace `code_hash`, set `expires_at = now() + 10 minutes`, `attempts = 0`, increment `resend_count`, set `last_sent_at`. Send the email. Return the two countdown values.

merit_lookup

1. Rate-limit (IP, member ID, and failed-lookup counter). Validate.
2. Find the member by `member_code`. If none, increment the failed-lookup counter and return `ID_NOT_FOUND`.
3. Return `first_name`, `total_points`, `attendance` (ledger rows with `source = event_checkin` joined to their events, newest first) and `adjustments` (rows with `source` of `manual_adjustment` or `check_in_reversal`, newest first).
4. Return nothing else. In particular no email, phone, last name or programme.

start_check_in and check_in

Specified in full in section 9.

submit_contact

1. Rate-limit. Validate. If `website` is not empty, return success and do nothing.
2. Verify the CAPTCHA token. Insert the `contact_messages` row.
3. The database webhook on insert calls `send_contact_notification`. Return `data: {}`.

send_contact_notification (webhook)

1. Reject the request unless the `x-webhook-secret` header equals `WEBHOOK_SECRET`.
2. Look up the email addresses of every user with the Communications Head or Technical Head role.
3. Email each of them the sender's name, email and message (section 11).

7.4 Admin functions: processing rules

Function	Processing	Audit action and details
`qr_token`	Load the event and token row; refuse `NOT_FOUND` if missing. Compute the token for the current window (section 9.2). Return it with `refresh_in_seconds`	None (it runs every 30 seconds)
`set_checkin_state`	Update `event_checkin_tokens.is_active`	`checkin_opened` or `checkin_closed`; `event_id`

Function	Processing	Audit action and details
`reset_event_secret`	Generate a new 256-bit hex secret. Expire every unused session for the event (`expires_at = now()`)	`event_secret_reset`; `event_id`
`manual_check_in`	Find member (else `ID_NOT_FOUND`); call `perform_manual_check_in`; map unique violation to `ALREADY_CHECKED_IN`	`manual_check_in`; `event_id, member_code`
`remove_check_in`	Call `perform_remove_check_in`	`check_in_removed`; `attendance_id, member_code, reason`
`adjust_merit`	Find member; insert ledger row `source = manual_adjustment` with reason and `awarded_by`; return the total	`merit_adjusted`; `member_code, points, reason`
`log_export`	Insert the audit row only	`registrations_exported`; `format, row_count`
`delete_event`	Refuse `EVENT_HAS_CHECKINS` if any attendance row exists. Delete the row (tokens cascade, linked media become unlinked) and the cover file	`event_deleted`; `event_id, title`
`delete_news_post`	Delete the row and the cover file	`news_deleted`; `post_id, title`
`delete_media_item`	Delete the row and, for pictures, the file at `storage_path`	`media_deleted`; `media_id, title`
`delete_member`	Call `perform_delete_member`	`member_deleted`; `member_code` only
`list_admins`	List auth users (`auth.admin.listUsers`, all pages) with their roles from `user_roles`	None
`invite_admin`	Call `auth.admin.inviteUserByEmail` with `redirectTo = SITE_URL + "/admin/accept-invite"`; if the user already exists skip the invite; insert the role	`admin_invited`; `email, role`
`set_user_role`	Add or remove the role. Refuse `LAST_TECHNICAL_HEAD` if the change would leave no Technical Head or removes the caller's own Technical Head role. Return the roles after the change	`role_changed`; `user_id, role, action`

Audit details must never contain coordinates, tokens, verification codes, phone numbers or email addresses of members. Each audit row stores `user_id`, `action`, `details` and `ip_address`.

8. Registration and the membership ID

8.1 Normalisation

```
export const normEmail = (e: string) => e.trim().toLowerCase();

export function normPhone(raw: string): string | null {
  const p = raw.replace(/[\s\-()]/g, '');
  if (/^(\+233|233|0)[2-9]\d{8}$/.test(p)) return '+233' + p.replace(/^(\+233|233|0)/, '');
  if (/^\+[1-9]\d{7,14}$/.test(p)) return p;
  return null;                                  // VALIDATION_ERROR on the phone field
}
```

8.2 Membership ID and code generation

- Generate the ID on the server from a cryptographically secure source, uniformly between 100000 and 999999. Do not derive it from the name and do not count upward.
- Uniqueness is enforced by `members_member_code_key`. On a unique violation of that constraint, generate a new number and retry, up to 5 times, then return `SERVER_ERROR`.
- The same generator pattern produces the 6-digit verification code, with a range of 000000 to 999999 padded to six digits.

```
export function randomInt(minInclusive: number, maxInclusive: number): number {
  const range = maxInclusive - minInclusive + 1;
  const limit = Math.floor(0x100000000 / range) * range;      // rejection sampling, no bias
  const buf = new Uint32Array(1);
  do { crypto.getRandomValues(buf); } while (buf[0] >= limit);
  return minInclusive + (buf[0] % range);
}
export const newMemberCode = () => String(randomInt(100000, 999999));
export const newVerificationCode = () => String(randomInt(0, 999999)).padStart(6, '0');
```

8.3 Rules

- A member exists only after the email is verified. Until then the registration is a pending row.
- The verification code is stored only as an HMAC, expires after 10 minutes, and allows 5 wrong attempts per code.
- The ID is returned once, by `verify_email`, and emailed so a lost ID can be recovered. Administrators can also see it on the Registrations screen.
- Registration collects personal data. Handling is described in section 12.3.

9. Event QR check-in engine

The QR on the projector is live, works only at the venue, and ties each check-in to the phone that made it. A photographed or shared link stops working within a minute, a phone outside the venue is refused, and one phone can check in only one member per event.

9.1 Flow

Step	Actor	Action
1	Organization Head	Creates the event. The `events_create_token` trigger creates its 256-bit secret. The head saves venue, radius and window
2	Admin QR screen	Calls `qr_token` now and every `refresh_in_seconds`
3	Member phone	Scans the QR, opens `/checkin/<token>`, gets a position, calls `start_check_in`
4	Backend	Verifies token, window, device and location. Opens a 3-minute single-use session bound to the device
5	Member	Enters the 6-digit ID. The phone calls `check_in` with the session, ID, device token and a fresh position
6	Backend	Runs the checks and records attendance and points in one transaction

9.2 The rotating token

- Window number: `floor(unix_seconds / rotation_seconds)` with `rotation_seconds = 30`.
- Token: `base64url(event_id + "." + window) + "." + base64url(HMAC-SHA256(secret, event_id + "." + window))`.

- A token is accepted only while the current window equals its window or the window after it, so it is good for at most 60 seconds. Nothing is written to the database when it rotates.
- `qr_token` returns `refresh_in_seconds = rotation_seconds - (now mod rotation_seconds)`, so the screen refreshes exactly at each window boundary.
- Compare signatures in constant time. Reject malformed tokens as `QR_EXPIRED`. Resetting the secret invalidates every token at once.

```
export function signToken(eventId: string, windowNo: number, secret: string): Promise<string> {
  const msg = eventId + '.' + windowNo;
  return hmacHex(secret, msg).then(mac => b64url(msg) + '.' + b64url(hexToBytes(mac)));
}

export async function verifyToken(token: string, secretFor: (eventId: string) => Promise<string |
null>,
                                  rotation = 30): Promise<string | null> {
  const [m, s] = token.split('.');
  if (!m || !s) return null;
  const msg = b64urlDecodeToString(m);
  const [eventId, w] = msg.split('.');
  const secret = await secretFor(eventId);
  if (!secret) return null;
  const expected = await hmacHex(secret, msg);
  if (!timingSafeEqualHex(expected, bytesToHex(b64urlDecodeToBytes(s)))) return null;
  const now = Math.floor(Date.now() / 1000 / rotation);
  const win = Number(w);
  return (now === win || now === win + 1) ? eventId : null;
}
```

9.3 start_check_in

1. Rate-limit by device (after hashing) and IP. Validate. Verify the token (9.2). Failure returns `QR_EXPIRED`.
2. Load the event and its token row. The event must be published, `is_active` must be true, and the time must lie within the check-in window (9.5). Otherwise `CHECKIN_CLOSED`.
3. Hash the device token: `HMAC-SHA256(DEVICE_HASH_PEPPER, device_token)` in hex.
4. Check the location (9.4). Failure returns `LOCATION_DENIED` or `OUTSIDE_AREA`.
5. Insert a `checkin_sessions` row with a random UUID, the event, the device hash and `expires_at = now() + 3 minutes`.
6. Return `session_id`, `expires_at`, `event_title` and `points`. Coordinates are discarded.

9.4 Location rules

- Measure the distance between the reported position and the venue with the Haversine formula.
- Accept only when the reported `accuracy` is 100 m or better and the distance is within the event's `radius_m`.
- Missing or invalid coordinates, or accuracy above 100 m, return `LOCATION_DENIED`. A distance above the radius returns `OUTSIDE_AREA`.
- Coordinates exist only in memory for the length of the request. They are never stored or logged. Only the result and the distance rounded to the nearest 10 m (`attendance.distance_m`) are kept.

```
export function haversineMeters(lat1: number, lng1: number, lat2: number, lng2: number): number {
  const R = 6371000, rad = (x: number) => x * Math.PI / 180;
  const dLat = rad(lat2 - lat1), dLng = rad(lng2 - lng1);
  const a = Math.sin(dLat / 2) ** 2 +
            Math.cos(rad(lat1)) * Math.cos(rad(lat2)) * Math.sin(dLng / 2) ** 2;
  return 2 * R * Math.asin(Math.sqrt(a));
}
// stored distance_m = Math.round(distance / 10) * 10
```

9.5 Check-in window

- Window opens at `checkin_opens_at` (or `starts_at` when null) and closes at `checkin_closes_at` (or `ends_at` when null) plus a 15-minute grace period.
- The window is evaluated on every call. No scheduled job is needed.
- An admin can close check-in at any time (`set_checkin_state`) or reset the secret (`reset_event_secret`).

9.6 check_in

1. Rate-limit by device, member ID and IP. Validate.
2. Increment the session's `attempts` and load it in one update. Return `SESSION_INVALID` when the session does not exist, is used, is expired, belongs to a different device hash, or `attempts` is above 5.
3. Check the location against the session's event (9.4).
4. Check the event is published, `is_active` is true and the time is inside the window (9.5). Otherwise `CHECKIN_CLOSED`.
5. Find the member by `member_code`. If none, count a failed lookup and return `ID_NOT_FOUND`. The session stays open for further attempts, up to 5.
6. Call `perform_check_in` with the distance rounded to 10 m, the hashed IP and the browser family.
7. Map a unique violation: constraint `attendance_event_member_uq` returns `ALREADY_CHECKED_IN`; `attendance_event_device_uq` returns `DEVICE_ALREADY_USED`. A `SESSION_INVALID` exception returns `SESSION_INVALID`. Because the function is one transaction, nothing is recorded on any failure.
8. Return `first_name`, `points_awarded`, `total_points` and `event_title`.

9.7 One phone, one check-in per event

- The browser keeps a random device token. The server stores only its HMAC with `DEVICE_HASH_PEPPER` in `checkin_sessions.device_hash` and `attendance.device_hash`.
- `attendance_event_member_uq` and `attendance_event_device_uq` make a second check-in impossible even under concurrent requests. Points are never awarded twice.
- A phone is not locked to one member across events in this release.

9.8 Review of bursts

Each QR check-in stores the hashed IP (`ip_hash`) and browser family (`ua_family`). The frontend flags five or more check-ins with the same pair within two minutes for an admin to review. The backend takes no automatic action, because many members legitimately share the campus network.

10. Merit ledger

- `merit_records` is an append-only ledger. A member's total is always the sum of their rows. No total is stored.
- Sources: `event_checkin` (the event's points, default 3), `manual_adjustment` (admin, with reason) and `check_in_reversal` (written when a check-in is removed).
- Rows are never updated or deleted by any function, except that deleting a member removes their rows.
- Removing a wrong check-in deletes the attendance row, writes a negative `check_in_reversal` row, and so lets the member check in again if appropriate.
- `merit_lookup` returns no personal data. The `leaderboard` view exposes display name, rank and points only, for members who opted in.

11. Email

All email is sent through Resend from `EMAIL_FROM`, with the display name “SPE UMaT Chapter”. Send each as plain text with a simple HTML alternative. Templates:

Email	Subject	Body
Verification code	Your SPE UMaT Chapter verification code	Hello <first_name>, your verification code is <code>. It expires in 10 minutes. If you did not try to register, ignore this email.
Membership ID	Your SPE UMaT Chapter membership ID	Welcome to the SPE UMaT Chapter, <first_name>. Your membership ID is <member_code>. Use it to check in to events and to see your merit points at <SITE_URL>/merit. Keep it safe and do not share it.
Contact notification	New message from <name>	From: <name> <email>. Message: <message>. Reply directly to the sender.
Admin invitation and password reset	Sent by Supabase Auth through Resend SMTP	Use the templates set in section 3.2

12. Security and privacy

12.1 Authentication and authorisation

- Supabase Auth for admins only. Roles in `user_roles`, checked with `has_role()`. Tokens expire hourly with refresh-token rotation.
- Roles are assigned only by the Technical Head through `invite_admin` and `set_user_role`. Every change is written to `audit_logs`. Roles are data, never hard-coded, so the yearly handover is a data change.

12.2 Public endpoint protection

- Rate limits per section 7.2, including per member ID on lookup and check-in, because a 6-digit ID has only 900,000 values.
- Generic errors for unknown IDs. Merit lookup returns only first name, points and history.
- Turnstile on registration and Contact Us, plus the honeypot.
- Zod validation on every input. Parameterised queries only. CORS limited to the approved origin.
- The frontend is served over TLS only. Functions are reachable only over HTTPS.

12.3 Personal data

- Registrations hold name, email, phone, programme and year. RLS blocks all public access. Only the Organization Head and Technical Head can read or export it.
- Every registrations export is recorded in `audit_logs` through `log_export`.
- Ghana’s Data Protection Act, 2012 (Act 843) applies. The frontend shows the notices in the Frontend Guide section 9.3. Location is checked and discarded. Only the result and a distance rounded to 10 m are kept.
- A member’s request for removal is handled by the Technical Head with `delete_member`.
- Logs and audit details contain no personal data, coordinates, tokens or codes.
- Daily automated backups are provided by the Supabase Pro plan. Perform one test restore into the staging project before launch.

12.4 Secret handling

- Secrets exist only in Supabase function secrets and GitHub Actions secrets.
- To rotate `DEVICE_HASH_PEPPER` accept that existing device hashes stop matching. This is harmless because device uniqueness applies per event.
- To rotate `CODE_HASH_PEPPER` do it outside live registrations, because pending codes become invalid.
- A leaked service role key is replaced in the Supabase dashboard and the secrets are reset immediately.

13. Scheduled jobs

Job	Schedule	Action
Cleanup	Daily at 02:00 UTC	Delete expired `rate_limits`, `checkin_sessions` and `email_verifications` older than one day
Database backup	Daily	Supabase Pro automated backup. Confirm it is enabled
Check-in windows	None	Evaluated on each call

```
select cron.schedule('daily-cleanup', '0 2 * * *', $$
  delete from rate_limits         where window_start < now() - interval '1 day';
  delete from checkin_sessions    where expires_at   < now() - interval '1 day';
  delete from email_verifications where expires_at   < now() - interval '1 day';
$$);
```

14. Deployment

14.1 Environments

Environment	Branch	Supabase project	Frontend
Production	`main`	Production (Pro plan)	Vercel production deployment
Staging	`develop`	Staging	Vercel preview deployment, configured with the staging environment variables

14.2 Pipeline

The frontend deploys through Vercel's GitHub integration on every push. Migrations and Edge Functions deploy from GitHub Actions. Pull requests run the frontend tests and build and the Deno function tests before merge.

```
name: Deploy backend
on:
  push:
    branches: [main, develop]
    paths: ['supabase/**']
jobs:
  deploy:
    runs-on: ubuntu-latest
    env:
      SUPABASE_ACCESS_TOKEN: ${{ secrets.SUPABASE_ACCESS_TOKEN }}
      SUPABASE_DB_PASSWORD:  ${{ github.ref_name == 'main' && secrets.PROD_DB_PASSWORD ||
secrets.STAGING_DB_PASSWORD }}
      PROJECT_REF:           ${{ github.ref_name == 'main' && secrets.PROD_PROJECT_REF ||
secrets.STAGING_PROJECT_REF }}
    steps:
      - uses: actions/checkout@v4
```

```
      - uses: supabase/setup-cli@v1
      - run: deno test --allow-env supabase/functions
      - run: supabase link --project-ref $PROJECT_REF
      - run: supabase db push
      - run: supabase functions deploy
```

- Run migrations only through the CLI (`supabase db push`), never by hand in the dashboard.
- GitHub secrets: `SUPABASE_ACCESS_TOKEN`, `PROD_PROJECT_REF`, `PROD_DB_PASSWORD`, `STAGING_PROJECT_REF`, `STAGING_DB_PASSWORD`.
- Function secrets are set with `supabase secrets set` per project and are never in the repository.
- Migrations must run cleanly on an empty project, so a new project can be rebuilt from the repository alone.

15. Test plan

Write automated tests for the shared helpers (normalisation, token signing and verification, Haversine, ID generation) and for each function's validation. Run the following against the staging project before every release.

Test	Expected result
Read `members`, `attendance`, `merit_records`, `event_checkin_tokens`, `audit_logs` with the anon key	Permission denied or no rows
Read `events` with the anon key	Permission denied. `events_public` returns published rows without venue or secret
Communications Head reads `members`	No rows
Communications Head changes an event's venue	Refused with `FORBIDDEN`
Register with an existing email or phone	`DUPLICATE_REGISTRATION`. No ID in the response
Honeypot filled	Normal-looking success; no row, no email
Wrong code 5 times, then the right code	`TOO_MANY_ATTEMPTS`
Right code after 10 minutes	`CODE_EXPIRED`
Resend twice within 60 seconds; resend a fourth time	`RATE_LIMITED` both times
Generate 100,000 IDs in a test	No ID outside 100000 to 999999. Collisions retried
Look up an unknown ID and a known ID with another member's details	Unknown IDs get an identical body; known IDs return only name, points and history
Sequential ID lookups from one IP	`RATE_LIMITED` within the limit in 7.2
QR token used after 60 seconds	`QR_EXPIRED`
QR token used after `reset_event_secret`	`QR_EXPIRED`; open sessions are also dead
Location outside the radius; accuracy 150 m; no location	`OUTSIDE_AREA`; `LOCATION_DENIED`; `LOCATION_DENIED`
Session used from another device token	`SESSION_INVALID`
Session used twice, after 3 minutes, or after 5 attempts	`SESSION_INVALID`
Same member checks in twice	`ALREADY_CHECKED_IN`; points unchanged
Second member on the same phone	`DEVICE_ALREADY_USED`
Two simultaneous check-ins for one member	One succeeds, one fails; points awarded once

Test	Expected result
Check-in after close, before opening, after the grace period, unpublished	`CHECKIN_CLOSED`
`delete_event` on an event with check-ins	`EVENT_HAS_CHECKINS`
`remove_check_in`	Attendance row gone, reversal row written, total correct
Each admin function with each wrong role	`FORBIDDEN`; nothing written
`log_export`	Audit row with user, format, count and time
`set_user_role` removing the only Technical Head	`LAST_TECHNICAL_HEAD`
Search function logs after a full check-in	No coordinates, tokens, device values, IDs, emails or phone numbers
Rebuild an empty project from migrations	Succeeds, and all tests above pass

16. Risks and controls

Risk	Control
A member's ID is used by someone else, a QR photo or link is shared, location is spoofed, or browser data is cleared to get a new device token	Rotating signed QR (30 seconds); single-use 3-minute session bound to the scanning phone; one check-in per phone per event; location check; check-in window; rate limits; review flag; admins can remove check-ins and reset the secret instantly
Guessing 6-digit IDs	Random IDs; rate limits per IP and per ID; generic errors; lookup returns only first name and points
Members lose their ID	The ID is emailed at registration and visible to admins
Duplicate or fake registrations	Unique email and phone; Turnstile; honeypot; rate limits; email verification
Personal data exposed	RLS on every table; default deny; no public reads; export limited to two roles and logged
GPS inaccurate indoors or location switched off	100 m default radius with margin; clear messages; manual check-in by admins; test in the real venue before launch
Executive handover each year	Roles in `user_roles`; Technical Head invites and removes admins; handover is a data change

17. Rulings

The following matters were decided in this specification. Neither developer needs to decide them again.

Matter	Ruling
API contract	The contract in section 4 is final. Request and response shapes, field names and error codes change only by a written change to both guides
Public function access	Public functions deploy with JWT verification off. Callers send the `apikey` header only. Admin functions keep JWT verification on and check roles in code
Admin role in the token	A Supabase custom access token hook adds an `app_roles` array claim, for example `["technical_head"]`. The frontend reads it for display. RLS and functions enforce

Matter	Ruling
Admin sign-in	Email and password. Public sign-up is disabled. The Technical Head invites admins through `invite_admin`. Invited admins set a password at `/admin/accept-invite`. Password reset uses `/admin/reset-password`. Minimum password length is 12 characters
First Technical Head	Created by the backend developer during setup (Backend section 3.5). All later admins are invited from the Admins screen
Permission split	As in the role matrix (section 8.2 of the Frontend guide, section 6.1 of the Backend guide). Only the Organization Head and Technical Head may export registrations
Export logging	Export is built in the browser. The frontend calls `log_export` first and builds the file only if the call succeeds
CAPTCHA	Cloudflare Turnstile on registration and Contact Us, together with a hidden honeypot field named `website`
Leaderboard	Included at `/leaderboard`. Shows the top 20 members who opted in. Opt-in checkbox on registration, unticked by default. Display name is first name and last initial
Merit adjustments	Included. Admin only, with a mandatory reason, logged. The reason is visible to the member
Event deletion	An event with check-ins cannot be deleted. Admins unpublish it instead
Check-in window	Defaults to the event's start and end time. The server accepts check-ins until 15 minutes after `checkin_closes_at`
Device lock across events	Not part of this release. The `devices` table is not created. One check-in per phone per event remains
Burst review	The server stores a hashed IP and browser family on each QR check-in. The Attendance screen flags 5 or more check-ins with the same pair within 2 minutes for review. Nothing is blocked automatically
Programme and year	Programme is a dropdown from `src/config/programmes.ts` plus "Other" with free text. Year is one of Year 1 to Year 4, or Postgraduate
Phone numbers	Ghana formats or international E.164 accepted. Stored as E.164
Email codes	6 digits, valid 10 minutes, 5 attempts, up to 3 resends with a 60-second cooldown
Data-protection wording	The wording in the notices table is final
Email sending	Resend. Sender address is the `EMAIL_FROM` secret. Staging uses Resend's test sender. Production requires the chapter's domain to be verified (prerequisite P5)
Venue privacy	Public pages read `events_public`, never `events`. Venue coordinates, radius and check-in times are never public
Rate limits	The thresholds in Backend section 7.2 are final for launch
Content format	News posts, event descriptions and site text are stored as Markdown and rendered with sanitisation
Time zone	Stored in UTC. Displayed in `Africa/Accra`
Hosting	Frontend on Vercel. Backend on Supabase. Production project on the Pro plan, which includes daily backups
Member data removal	Technical Head uses `delete_member` on request. It is logged
Contact notifications	A database webhook calls `send_contact_notification`, which emails every Communications Head and Technical Head

18. Prerequisites and build order

18.1 Prerequisites

These are accounts and materials the chapter provides. They are not design decisions. Work that depends on an item is blocked until it is supplied.

Ref	Item	Supplied by	Needed for
P1	Supabase production project (Pro plan) and a separate staging project	Technical Head	All backend work
P2	GitHub repository with the two developers as collaborators	Technical Head	All work
P3	Vercel project linked to the repository	Technical Head	Frontend deployment
P4	Domain name and DNS access	Technical Head	Production launch, email
P5	Resend account, API key, and the sending domain verified with DNS records	Technical Head	Verification emails in production
P6	Cloudflare Turnstile site (site key and secret key)	Technical Head	Registration and Contact Us
P7	Email address of the first Technical Head	Technical Head	Backend bootstrap
P8	Launch content and brand assets: logos, brand colours, photos, executives' names, positions and pictures, About text, benefits text, contact details	Communications Head	Content entry before launch

18.2 Build order

Phase	Backend delivers	Frontend delivers	Joint checkpoint
1. Foundation	Projects, schema, RLS, auth hook, storage, seed data, first Technical Head, staging	Vite app, routing, layouts, Supabase client, API client, shared types, public pages reading seed data	Public pages render from staging. Anon key cannot read private tables
2. Registration and merit	`register_member`, `verify_email`, `resend_code`, `merit_lookup`, email templates	Registration, verification, success and merit pages	A real registration end to end. Duplicate and code errors behave as specified
3. Admin core	Admin functions, invites, roles, audit log, views	Login, invite acceptance, role-gated shell, Registrations with export, Admins, Audit log	Each of the three roles sees exactly the screens in the role matrix
4. Events and check-in	`start_check_in`, `check_in`, `qr_token`, check-in database function, venue trigger	Events screen, live QR screen, check-in pages, Attendance, Merit Tracker	A full check-in in the real venue on real phones, including a refusal
5. Content and launch	Scheduled jobs, backups confirmed, production deployment, rate-limit tuning	News, Media and Content, Messages, leaderboard, performance and accessibility pass	Acceptance criteria met. Launch content loaded. Production smoke test passed

Appendix A. Database schema

> **Authoritative** This schema is identical in the Frontend Guide. Section 5 adds the constraint names, triggers, views and functions.

This schema is final for both developers. Every table has `created_at` (default `now()`) and `updated_at` unless stated. All timestamps are `timestamptz`. Backend owns migrations and RLS. Frontend generates its types from this schema.

```
members
  id uuid PK default gen_random_uuid()
  first_name text not null, last_name text not null
  email text not null unique            -- lower case
  phone text not null unique            -- E.164
  programme text not null
  year_of_study text not null           -- Year 1 | Year 2 | Year 3 | Year 4 | Postgraduate
  member_code char(6) not null unique   -- check (member_code ~ '^[1-9][0-9]{5}$')
  show_on_leaderboard boolean not null default false
  email_verified_at timestamptz not null

email_verifications                     -- pending registrations
  id uuid PK
  email text not null unique            -- one pending row per email
  registration_data jsonb not null      -- first_name, last_name, programme,
                                        -- year_of_study, phone, show_on_leaderboard
  code_hash text not null               -- HMAC-SHA256, never the code itself
  expires_at timestamptz not null       -- 10 minutes after the latest send
  attempts integer not null default 0   -- wrong codes, maximum 5
  resend_count integer not null default 0   -- maximum 3
  last_sent_at timestamptz not null

user_roles                              -- admins only
  user_id uuid FK -> auth.users(id) on delete cascade
  role app_role not null                -- communications_head | organization_head | technical_head
  PK (user_id, role)

site_content
  key text PK                           -- home_intro, about_mission, about_vision,
                                        -- about_benefits, contact_info
  body text not null                    -- Markdown

events
  id uuid PK
  title text not null, slug text not null unique
  description text not null             -- Markdown
  starts_at timestamptz not null, ends_at timestamptz not null
  location text not null
  cover_image_url text
  points integer not null default 3
  checkin_opens_at timestamptz, checkin_closes_at timestamptz
  venue_lat numeric(9,6), venue_lng numeric(9,6)
  radius_m integer not null default 100 -- check (radius_m between 20 and 1000)
  is_published boolean not null default false

event_checkin_tokens                    -- never public; created by a trigger on events
  event_id uuid PK FK -> events(id) on delete cascade
  secret text not null                  -- 256-bit random, hex
  is_active boolean not null default true
  rotation_seconds integer not null default 30

attendance
  id uuid PK
  event_id uuid not null FK -> events(id) on delete restrict
  member_id uuid not null FK -> members(id) on delete restrict
  checked_in_at timestamptz not null default now()
  method text not null                  -- qr | manual
```

```
  device_hash text                      -- null for manual
  distance_m integer                    -- rounded to 10 m; null for manual
  ip_hash text, ua_family text          -- for review of check-in bursts; null for manual
  UNIQUE (event_id, member_id)
  UNIQUE (event_id, device_hash)

merit_records                           -- points ledger, append only
  id uuid PK
  member_id uuid not null FK -> members(id) on delete restrict
  event_id uuid FK -> events(id) on delete restrict
  attendance_id uuid FK -> attendance(id) on delete set null
  points integer not null
  source text not null                  -- event_checkin | manual_adjustment | check_in_reversal
  reason text
  awarded_by uuid FK -> auth.users(id)

news_posts
  id uuid PK
  title text not null, slug text not null unique
  body text not null                    -- Markdown
  cover_image_url text
  is_published boolean not null default false
  published_at timestamptz
  created_by uuid FK -> auth.users(id)

media_items
  id uuid PK
  type text not null                    -- youtube | recording | photo
  title text not null
  category text not null default 'general'  -- photos: general | event | activity | member |
executive
  url text                              -- YouTube or recording link; null for photos
  storage_path text                     -- path in bucket site-media; null for links
  event_id uuid FK -> events(id) on delete set null
  is_published boolean not null default true

contact_messages
  id uuid PK
  name text not null, email text not null, message text not null
  is_read boolean not null default false

audit_logs                              -- append only
  id uuid PK
  user_id uuid FK -> auth.users(id)
  action text not null
  details jsonb not null default '{}'
  ip_address inet
  created_at timestamptz not null default now()

checkin_sessions
  id uuid PK default gen_random_uuid()
  event_id uuid not null FK -> events(id) on delete cascade
  device_hash text not null
  expires_at timestamptz not null       -- 3 minutes after the scan
  used_at timestamptz
  attempts integer not null default 0   -- maximum 5

rate_limits
  key text PK
  window_start timestamptz not null
  count integer not null
```

Views

View	Readable by	Columns
`events_public`	Everyone	`id, title, slug, description, starts_at, ends_at, location, cover_image_url, points` for published events only. Hides venue, radius and check-in times
`leaderboard`	Everyone	`display_name` (first name and last initial, for example "Kwame A."), `total_points`, `rank`. Only members with `show_on_leaderboard = true`. Ranked among those members
`admin_registrations`	Organization Head, Technical Head	`id, first_name, last_name, email, phone, programme, year_of_study, member_code, show_on_leaderboard, created_at, total_points`
`admin_attendance`	Organization Head, Technical Head	`attendance_id, event_id, member_id, member_code, first_name, last_name, checked_in_at, method, distance_m, ip_hash, ua_family`
`admin_merit`	Organization Head, Technical Head	`id, member_id, member_code, first_name, last_name, event_id, event_title, points, source, reason, awarded_by, created_at`

The three `admin_*` views are created with `security_invoker = true`, so RLS on the base tables applies to the caller. `events_public` and `leaderboard` run as the view owner and are granted to `anon`.

Appendix B. API types

The frontend copies this file unchanged. Your Zod schemas must accept exactly these shapes.

```
// api-types.ts  (identical file in the frontend repo; backend validates with matching Zod schemas)
export type ErrorCode =
  | 'VALIDATION_ERROR' | 'CAPTCHA_FAILED' | 'CODE_INVALID' | 'LOCATION_DENIED'
  | 'UNAUTHORIZED' | 'FORBIDDEN' | 'CHECKIN_CLOSED' | 'OUTSIDE_AREA' | 'ID_NOT_FOUND'
  | 'NOT_FOUND' | 'DUPLICATE_REGISTRATION' | 'ALREADY_CHECKED_IN' | 'DEVICE_ALREADY_USED'
  | 'EVENT_HAS_CHECKINS' | 'LAST_TECHNICAL_HEAD' | 'CODE_EXPIRED' | 'QR_EXPIRED'
  | 'SESSION_INVALID' | 'RATE_LIMITED' | 'TOO_MANY_ATTEMPTS' | 'SERVER_ERROR';

export type ApiResponse<T> =
  | { ok: true; data: T }
  | { ok: false; error: { code: ErrorCode; message: string;
      details?: { field: string; message: string }[]; retry_after_seconds?: number } };

export type AppRole = 'communications_head' | 'organization_head' | 'technical_head';
export type YearOfStudy = 'Year 1' | 'Year 2' | 'Year 3' | 'Year 4' | 'Postgraduate';
export interface Location { lat: number; lng: number; accuracy: number }

export interface RegisterMemberReq {
  first_name: string; last_name: string; programme: string; year_of_study: YearOfStudy;
  email: string; phone: string; show_on_leaderboard?: boolean;
  captcha_token: string; website: string;
}
export interface RegisterMemberRes { email: string; expires_in_seconds: number;
resend_available_in_seconds: number }
export interface VerifyEmailReq { email: string; code: string }
export interface VerifyEmailRes { member_code: string; first_name: string }
export interface ResendCodeReq { email: string }
export type ResendCodeRes = Omit<RegisterMemberRes, 'email'>;

export interface MeritLookupReq { member_code: string }
export interface MeritLookupRes {
  first_name: string; total_points: number;
  attendance: { event_title: string; event_date: string; points: number; checked_in_at: string }[];
  adjustments: { points: number; reason: string; created_at: string }[];
```

```
}

export interface StartCheckInReq { event_token: string; device_token: string; location: Location }
export interface StartCheckInRes { session_id: string; expires_at: string; event_title: string;
points: number }
export interface CheckInReq { session_id: string; member_code: string; device_token: string;
location: Location }
export interface CheckInRes { first_name: string; points_awarded: number; total_points: number;
event_title: string }

export interface SubmitContactReq { name: string; email: string; message: string; captcha_token:
string; website: string }

export interface QrTokenReq { event_id: string }
export interface QrTokenRes { token: string; refresh_in_seconds: number }
export interface SetCheckinStateReq { event_id: string; is_active: boolean }
export interface ManualCheckInReq { event_id: string; member_code: string }
export type ManualCheckInRes = Omit<CheckInRes, 'event_title'>;
export interface RemoveCheckInReq { attendance_id: string; reason: string }
export interface AdjustMeritReq { member_code: string; points: number; reason: string }
export interface LogExportReq { format: 'csv' | 'xlsx'; row_count: number }
export interface AdminSummary { user_id: string; email: string; roles: AppRole[]; last_sign_in_at:
string | null }
export interface InviteAdminReq { email: string; role: AppRole }
export interface SetUserRoleReq { user_id: string; role: AppRole; action: 'add' | 'remove' }
```